Валидация «Взаимно»: результаты

> Файл сгенерирован командой `cd backend && uv run python -m eval.validate` — режим полный, сид `20261005`, коммит `50538b0f9d`, 471 с на 4 процессах. Руками не редактируется. Протокол — VALIDATION_PLAN.md: метрики и пороги в нём записаны до прогона, уточнения процедуры после прогонов вынесены в отдельные разделы 4–5 (пороги не менялись). Все числа в машиночитаемом виде — `../backend/eval/out/validation.json` (его же показывает «Лаборатория» после `--save-db`).

Всё ниже — **симуляция**: она проверяет процедуру на синтетической популяции с известной истиной и работает на тех же движках, что и прод (`app/testing`, банк из 102 семейств с экспертными оценками сложности, `app/matching`, `app/fsp`). Раздела «люди» (пилот, экспертная разметка пар) в этом прогоне нет — см. «Честные ограничения».

Итоги

Раздел	Метрика	Значение	Цель	Статус
Тестирование	Верное решение у заявленного порога	0,86	≥ 0,85	✓ выполнено
Тестирование	Точный грейд	0,748	≥ 0,75	✗ не выполнено
Тестирование	Грейд с точностью ±1	1,00	≥ 0,99	✓ выполнено
Тестирование	Средняя длина основной части, заданий	12,8	≤ 15,0	✓ выполнено
Тестирование	Средняя длина с доборами, заданий	16,1	≤ 17,0	✓ выполнено
Тестирование	p95 длины с доборами, заданий	20,0	≤ 20,0	✓ выполнено
Тестирование	Показано в основной части вместе с pretest (максимум; pretest-семейств в банке пока нет)	20	≤ 20	✓ выполнено
Тестирование	Пересдача: совпадение решения	0,81	≥ 0,78	✓ выполнено
Тестирование	Пересдача: к решения	0,553	≥ 0,55	✓ выполнено
Тестирование	Пересдача: QWK грейда	0,86	≥ 0,85	✓ выполнено
Тестирование	Пересдача: корреляция оценок уровня	0,86	≥ 0,88	✗ не выполнено
Тестирование	Ложное повышение грейда (гистерезис)	0,03	≤ 0,07	✓ выполнено
Тестирование	Общих экземпляров у пар кандидатов (каждый экземпляр выдаётся одному — по построению)	0	0	✓ выполнено
Тестирование	Двум кандидатам с одним семейством выпал одинаковый по тексту экземпляр (Backend, худшее семейство)	0,10%	показываем	справочно
Тестирование	RMSE сложности после 100 ответов	0,20	≤ 0,30	✓ выполнено
Тестирование	Теги ✓ модулей: точность против истинного уровня (ориентир концепции ≥ 0,90)	0,97	показываем	справочно
Тестирование	Теги ✓ модулей: полнота (ориентир концепции ≥ 0,75)	0,76	показываем	справочно
Утечки	Ложный Middle у явных Junior, коалиция 20 (худшая стратегия списывания)	0,10	≤ 0,15	✓ выполнено
Утечки	Наивные списывающие пойманы «чужим ключом» (коалиция 20)	1,00	≥ 0,90	✓ выполнено
Утечки	B0 фиксированная форма: ложный Middle при коалиции 20	1,00	показываем	справочно
Дрейф	CUSUM, 50% знают метод: медиана задержки (прод-порог h), ответов	603	≤ 200	✗ не выполнено

Раздел	Метрика	Значение	Цель	Статус
Дрейф	CUSUM: ложные карантины без утечки (прод-порог h)	0,01	≤ 0,05	✓ выполнено
Категоризация	macro-F1 подсказки специализации (V-CAT)	0,93	≥ 0,90	✓ выполнено
Категоризация	Профили без ФСП получают категорию по тем же правилам	да	да	✓ выполнено
Подбор	P@10 по оракулу, test-брифы, сквозной режим	0,61	≥ 0,60	✓ выполнено
Подбор	P@10 значимо выше B2 (парный бутстреп)	нет	да	✗ не выполнено
Подбор	P@10 значимо выше B2 на всех 40 брифах (постфактум-срез)	да	да	✓ выполнено
Разбор брифа	Специализация top-1	0,90	≥ 0,90	✓ выполнено
Разбор брифа	Грейд точно	0,95	≥ 0,75	✓ выполнено
Разбор брифа	Грейд ±1	1,00	≥ 0,95	✓ выполнено
Разбор брифа	F1 навыков	0,98	≥ 0,85	✓ выполнено
Разбор брифа	must/nice определены верно	0,81	≥ 0,80	✓ выполнено
Разбор брифа	Верная категория в топ-3	0,95	≥ 0,95	✓ выполнено
Устойчивость	Стабильность топ-10 при пересдаче (overlap@10)	0,28	≥ 0,70	✗ не выполнено
Устойчивость	P@10 после пересдачи (постфактум): качество десятки не зависит от шума теста	0,68	≥ 0,60	✓ выполнено
Устойчивость	Стабильность топ-10 / потолок ранжирования по одному измерению уровня при той же надёжности (постфактум)	3,3	≥ 1,0	✓ выполнено
Устойчивость	Жаккар топ-10 после «добавлен nice-навык»	0,96	≥ 0,60	✓ выполнено
Устойчивость	Падение P@10 при сдвиге веса на ±0,05 (максимум)	0,01	≤ 0,05	✓ выполнено
Устойчивость	Накрутка навыков: сдвиг ранга вверх, наш подбор	0,0	≤ 2,0	✓ выполнено
Устойчивость	Накрутка навыков: сдвиг ранга вверх, BM25 (ожидание ≥ 10)	522,5	≥ 10,0	✓ выполнено
Экспозиция	Кандидаты без ФСП: условное отношение экспозиции, топ-10	0,92	0,80–1,25	✓ выполнено
Экспозиция	Кандидаты без ФСП: условное отношение экспозиции, топ-50	0,94	0,80–1,25	✓ выполнено
Объяснения	Σ вкладов = Score	1,00	≥ 1,00	✓ выполнено
Объяснения	Доводы ссылаются на существующие доказательства	1,00	≥ 1,00	✓ выполнено
Объяснения	Отсутствие ФСП никогда не «против»	1,00	≥ 1,00	✓ выполнено
Объяснения	Без доказательства довода №1 Score падает не меньше превышения над медианой	1,00	≥ 1,00	✓ выполнено
Производительность	p95 сборки версии подборки, мс	215	≤ 300	✓ выполнено

Выполнено 38 из 43 пороговых пунктов. Не выполнены: «Точный грейд», «Пересдача: корреляция оценок уровня», «CUSUM, 50% знают метод: медиана задержки (прод-порог h), ответов», «P@10 значимо выше B2 (парный бутстреп)», «Стабильность топ-10 при пересдаче (overlap@10)».

1. Данные

Популяция с латентной истиной (`eval/population.py`). Каждый кандидат порождается своим генератором с сидом, поэтому популяция из n кандидатов — префикс популяции из 2n.

Показатель	Значение
Кандидатов	2 000
Истинная специализация	Backend (Python) 50%, Frontend (JS/TS) 39%, Fullstack 10%

Показатель	Значение
Истинный грейд	Ниже Стажёра 5%, Стажёр 16%, Junior 30%, Middle 31%, Senior 19%
Заявленная специализация не совпадает с истинной	9,5%
Есть история ФСП	35,4%
Корреляция VSS (лучшая дисциплина) с истинным уровнем	0,48 (Спирмен 0,51, n = 708)
Есть регулярные задачи	49,4%
Категория подтверждена тестом	93,2%
Навыков в истинном стеке / приписано в самоописании (в среднем)	14,5 / 4,3

Брифы (`eval/briefs/*.json`): 40 русскоязычных брифов, разбиение 20 dev / 20 test зафиксировано в файлах. Стили: переписка со сленгом и опечатками — 11, формальная вакансия — 8, длинное описание с разделами — 7, смесь с английским — 6, короткий запрос — 8. **Тексты и эталонная разметка (специализация, грейд, must/nice, формат, город, вилка) написаны командой с помощью ИИ-ассистента**; посторонних авторов в наборе нет, а история правок эталона до первого коммита не зафиксирована. Все итоговые цифры подбора — по test-брифам; веса на брифах не подбирались.

2. Тестирование

Метрики точности и пересдачи — по кандидатам, чья заявленная специализация совпадает с истинной (n = 1 810). Решение «верное», если «сдал» ⇔ истинный уровень не ниже порога заявленного грейда. «Не подтверждён» считается как «Ниже Стажёра».

2.1. Точность

Система	Верное решение у порога	Точный грейд	±1	к	QWK
Наш CAT (основная часть + доборы)	0,86	0,75	1,00	0,67	0,90
B4: грейд по стажу (подсказка опроса)	—	0,54	0,97	0,38	0,75

Точность решения по тому, как кандидат заявил грейд: верно — 0,84 (n = 1243); на ступень выше — 0,85 (n = 420); на ступень ниже — 1,00 (n = 147).

Матрица «истинный грейд → итоговый грейд» (строки — истина):

Истина \ итог	Ниже Стажёра / не подтверждён	Стажёр	Junior	Middle	Senior
Ниже Стажёра / не подтверждён	62	31	0	0	0
Стажёр	34	205	29	0	0
Junior	0	85	400	56	0
Middle	0	1	105	421	35
Senior	0	0	1	80	265

Точность по специализациям теста (кандидаты с верной заявкой):

Специализация	n	Верное решение	Точный грейд	±1	Подтверждено	Заданий в тесте: среднее / p95
backend_python	1003	0,86	0,73	1,00	94%	15,8 / 20
frontend_js	807	0,86	0,76	1,00	95%	16,5 / 20

Кандидаты, ошибившиеся со специализацией (проходят чужой тест, n = 190): категорию получают 79%, на уровне своего истинного грейда или выше — 24%. Подсказка опроса (раздел 3) нужна, чтобы таких было меньше.

2.2. Длина теста

Метрика	Значение	Цель
Основная часть, оцениваемых заданий: среднее / p95	12,8 / 20	≤ 15
С доборами, оцениваемых: среднее / p95	16,1 / 20	≤ 17 / ≤ 20
Основная часть, показано вместе с pretest: среднее / p95 / максимум	12,8 / 20 / 20	≤ 20 (DECISIONS п.17)
Pretest-заданий на сессию	0,00	—
Доля основных частей, дошедших до максимума	30%	—

Решения основной части: не сдал, на границе — 204, сдал на границе — 328, не сдал — 366, сдал — 912.

2.3. Пересдача и гистерезис

Вторая сессия того же человека; все семейства первой сессии исключены (n = 1 810).

Метрика	Значение	Цель
Совпадение решения	0,81	≥ 0,78
к решения	0,55	≥ 0,55
Точное совпадение грейда / ±1	0,68 / 0,99	показываем
QWK грейда	0,86	≥ 0,85
Корреляция оценок уровня	0,86	≥ 0,88
Ложное повышение при тесте на грейд выше (гистерезис)	0,03 (n = 400)	≤ 0,07

2.4. Ошибка экспертов в сложности и крутизне (S2)

Истинная сложность = экспертная + $N(0; \sigma^2)$ — ошибка одна на семейство; истинная крутизна A ≠ 1,5 модели. n = 400 кандидатов на условие.

Условие	Верное решение у порога	Точный грейд
σ = 0, A = 1,5 (модель верна)	0,87	0,77
σ = 0,3	0,85	0,76
σ = 0,5	0,87	0,76
истинная крутизна A = 1	0,81	0,68
истинная крутизна A = 2	0,88	0,78

2.5. Бейзлайны тестирования

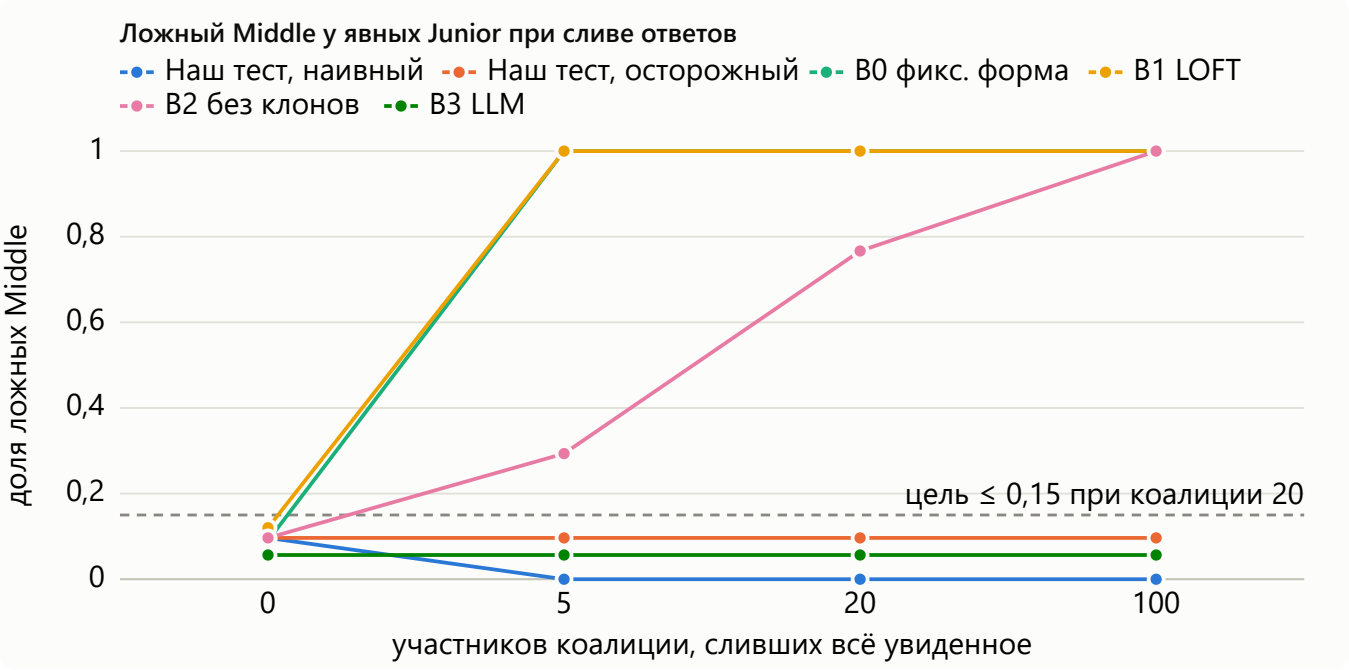
Подвыборка n = 400. B0 и B1 — формы из 16 заданий («сдал», если вероятность уровня не ниже порога ≥ 0,5). B3 «LLM под каждого»: каждое задание сгенерировано заново, его сложность = задуманная + $N(0; \sigma^2)$ на экземпляр, часть ключей неверна. **Доля битых ключей — допущение, а не замер**: реального прогона заданий через YandexGPT/GigaChat в этом прогоне нет; поэтому показана чувствительность к этой доле.

Система	Верное решение	Точный грейд	Длина	Совпадение решения при пересдаче
Наш CAT (клоны)	0,85	0,73	12,7	0,79
B0 фиксированная форма (16)	0,84	0,68	16	—

Система	Верное решение	Точный грейд	Длина	Совпадение решения при пересдаче
B1 LOFT (16)	0,85	0,68	16	0,81
B3 «LLM под каждого» (σ = 0,5, битых ключей 10%)	0,77	0,66	13,6	0,71
B3, битых ключей 0%	0,86	0,76	13,3	—
B3, битых ключей 5%	0,82	0,66	13,3	—
B3, битых ключей 20%	0,63	0,45	13,5	—
B3 «идеальный LLM» (σ = 0,3, 0% битых)	0,87	0,76	13,2	0,79

2.6. Утечки (S4)

300 «явных Junior» (истинный уровень равномерно в [−0,8; −0,3]) заявляют Middle (Backend). Коалиция из k честных кандидатов сливает всё, что видела. Наш тест: экземпляры одноразовые; «наивный» списывающий вводит слитый ключ чужого экземпляра того же семейства, «осторожный» видит, что код другой, и отвечает честно. B0/B1/B2: копирует, только если достался тот же экземпляр (консервативно в пользу бейзлайнов). B3: экземпляры уникальны, слив бесполезен — его доля ложных Middle равна честной ошибке.



Коалиция	0	5	20	100
Слито семейств банка Backend	0%	77%	95%	100%
Наш тест, наивный списывающий	0,10	0,00	0,00	0,00
Наш тест, осторожный	0,10	0,10	0,10	0,10
B0 фиксированная форма	0,09	1,00	1,00	1,00
B1 LOFT	0,12	1,00	1,00	1,00
B2 CAT без клонов (20 экземпляров)	0,10	0,29	0,77	1,00
B3 «LLM под каждого»	0,06	0,06	0,06	0,06
Наивные пойманы «чужим ключом»	—	1,00	1,00	1,00

В «Итогах» для нашего теста берётся худшая из двух стратегий списывающего: наивный ввод чужого ключа почти всегда проваливается, поэтому рациональный списывающий отвечает честно, и его доля ложных Middle равна честной ошибке теста у порога.

Утечка метода (коалиция 20, слито 95% семейств): тем, кто знает идею семейства, оно легче на Δb. Клоны защищают от слива ответов, но не от слива идеи — это видно в таблице.

Δb	Ложный Middle
0,00	0,10
0,25	0,25
0,50	0,51

Детектор «чужой ключ» активен для 28 из 44 семейств Backend (доля коллизий ключей < 2%, измерено на 200 экземплярах каждого семейства реального банка; медиана коллизий 0,00). У семейств с выбором ответа ключи совпадают часто, поэтому для них детектор не работает — их защищают только клоны и CUSUM.

2.7. Дрейф: CUSUM (S5)

Поток ответов по каждому из 44 семейств Backend: сначала объём 3 000 честных сессий, затем столько же, где доля проходящих знает метод 12 семейств (Δb = 0,5). Повторов: 20. Прод-порог h = 11.



Сценарий	h	Найдено слитых семейств	Медиана задержки, ответов	Ложные карантины
без утечки	4	—	—	0,633
без утечки	5	—	—	0,380
без утечки	6	—	—	0,204
без утечки	7	—	—	0,120
без утечки	8	—	—	0,060
без утечки	9	—	—	0,032
без утечки	10	—	—	0,018
без утечки	11	—	—	0,011
без утечки	12	—	—	0,004
20% знают метод	4	0,33	281	0,548
20% знают метод	5	0,33	426	0,306
20% знают метод	6	0,20	441	0,172
20% знают метод	7	0,16	436	0,091

Сценарий	h	Найдено слитых семейств	Медиана задержки, ответов	Ложные карантины
20% знают метод	8	0,12	513	0,057
20% знают метод	9	0,10	520	0,035
20% знают метод	10	0,07	557	0,016
20% знают метод	11	0,07	615	0,008
20% знают метод	12	0,03	648	0,006
50% знают метод	4	0,52	157	0,552
50% знают метод	5	0,70	226	0,312
50% знают метод	6	0,67	306	0,184
50% знают метод	7	0,66	370	0,101
50% знают метод	8	0,57	421	0,049
50% знают метод	9	0,50	461	0,028
50% знают метод	10	0,44	556	0,014
50% знают метод	11	0,35	603	0,008
50% знают метод	12	0,30	703	0,004

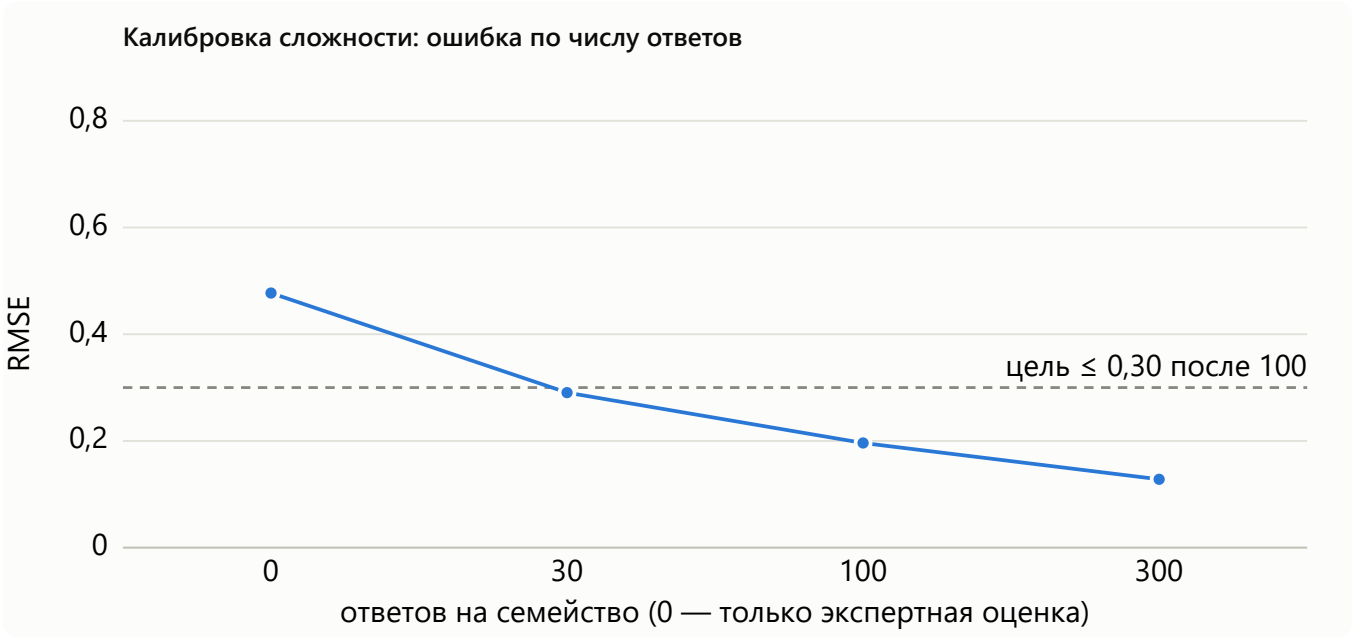
Подбор прод-порога на других популяциях. Порог h выбирается по нулевым потокам (без утечки) на отдельных сидах, а не на основном; основной сид лишь проверяет выбор. Ложные карантины по h:

Популяция	4	5	6	7	8	9	10	11	12
сид 301 (подбор)	0,690	0,449	0,293	0,177	0,108	0,070	0,050	0,027	0,017
сид 302 (подбор)	0,701	0,481	0,314	0,189	0,103	0,064	0,039	0,023	0,017
сид 303 (подбор)	0,693	0,473	0,330	0,201	0,124	0,081	0,053	0,026	0,018
основной сид (проверка)	0,633	0,380	0,204	0,120	0,060	0,032	0,018	0,011	0,004

Наименьший h с ложными ≤ 5% на каждом из сидов подбора — 11; прод-порог h = 11 даёт на основном сиде 1% ложных карантинов (цель ≤ 5%). При 50% знающих такой порог находит 35% слитых семейств с медианой задержки 603 ответа — цель ≤ 200 ответов **не достигнута**: задержка ≈ h / (информация на ответ), а при Δb = 0,5 и половине знающих эффект на один ответ мал (порядка 0,02–0,03 нат); уложиться в 200 ответов можно только ценой ложных карантинов 12–31% (h = 5–7 в таблице) или при вдвое большем трафике на семейство. Выбран контроль ложных карантинов: без утечки CUSUM — сигнал методисту, а не автоматическая блокировка. Порог плана ≤ 200 не меняем.

2.8. Калибровка сложности

Истинная сложность неякорных семейств = экспертная + $N(0; 0{,}5^2)$; MAP-уточнение `calibration.calibrate_family` по n ответам. Семейств: 25, повторов: 5.



Ответов на семейство	0	30	100	300
RMSE	0,48	0,29	0,20	0,13

2.9. Дискриминативность

r_pb — точечно-бисериальная корреляция ответа с оценкой уровня **без этого задания**; pretest-семейств в банке пока нет, поэтому считаем по боевым заданиям. ΔP — доля верных у Middle минус у Junior (по оценке уровня).

Специализация	Семейств	Медиана r_pb	Доля r_pb ≥ 0,20	Флаг r_pb < 0,10	Медиана ΔP	Доля ΔP ≥ 0,20
backend_python	44	0,44	0,98	py.closures.late_binding	0,20	0,46
frontend_js	40	0,46	0,97	нет	0,22	0,63

AUC оценки уровня для соседних грейдов: intern / junior — 0,91, junior / middle — 0,88, middle / senior — 0,90. Надёжность 1 − mean(SE^2)/var: 0,83.

Пересечение экземпляров у пар кандидатов: 0 из 67 929 выданных (тест и пересдача). Это свойство выдачи, а не измерение: симулятор, как и пул в проде, отдаёт каждый экземпляр одному человеку. Измеримый вопрос — увидят ли двое одно и то же задание под разными id: по 200 экземплярам каждого из 44 семейств Backend вероятность, что двум кандидатам с одним семейством выпадет одинаковый по тексту экземпляр, в среднем 0,00%, в худшем семействе (py.complexity.containers) — 0,10%; совпадения есть у 2 семейств. У таких близнецов и ключ общий; модель утечек в 2.6 это учитывает: слитый ключ верен с этой вероятностью (плюс угадывание).

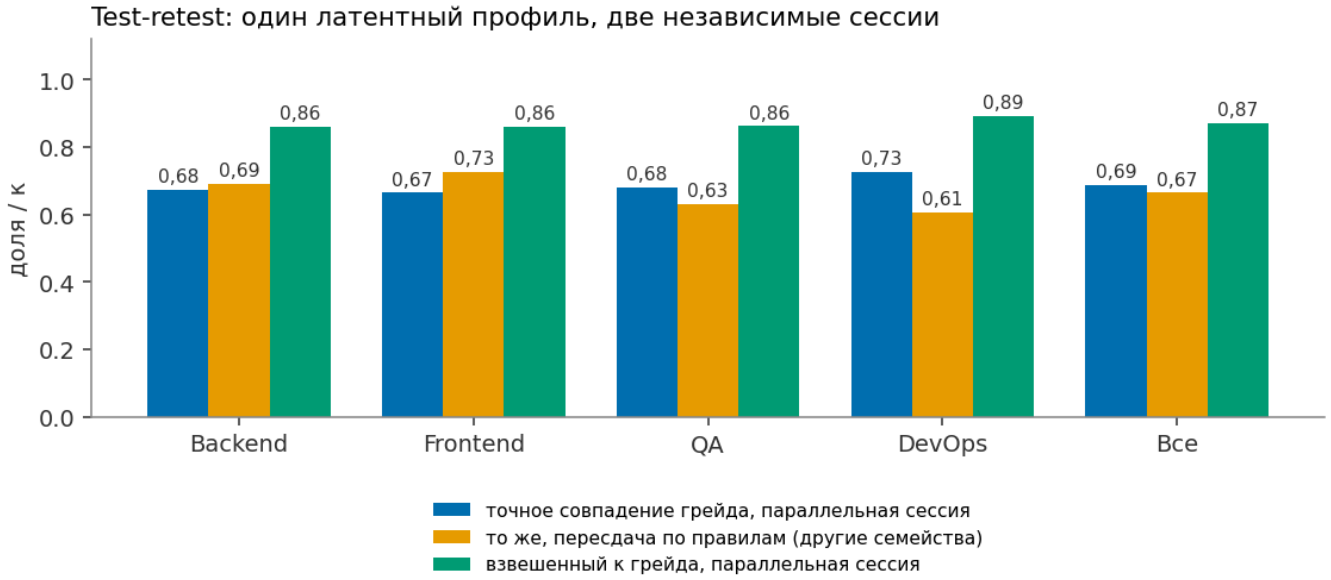
Теги модулей. ✓ утверждает «уровень модуля не ниже порога Junior», ✓✓ — «не ниже порога итогового грейда»; истина — уровень модуля генератора. По 8 570 модулям: точность ✓ 0,97, полнота 0,76 (полнота считается по всем модулям теста, у 10% из них заданий меньше двух — «~»); точность ✓✓ 0,91 (n = 4224). В плане этой метрики не было, поэтому ориентир концепции (≥ 0,90 / ≥ 0,75) показан справочно.

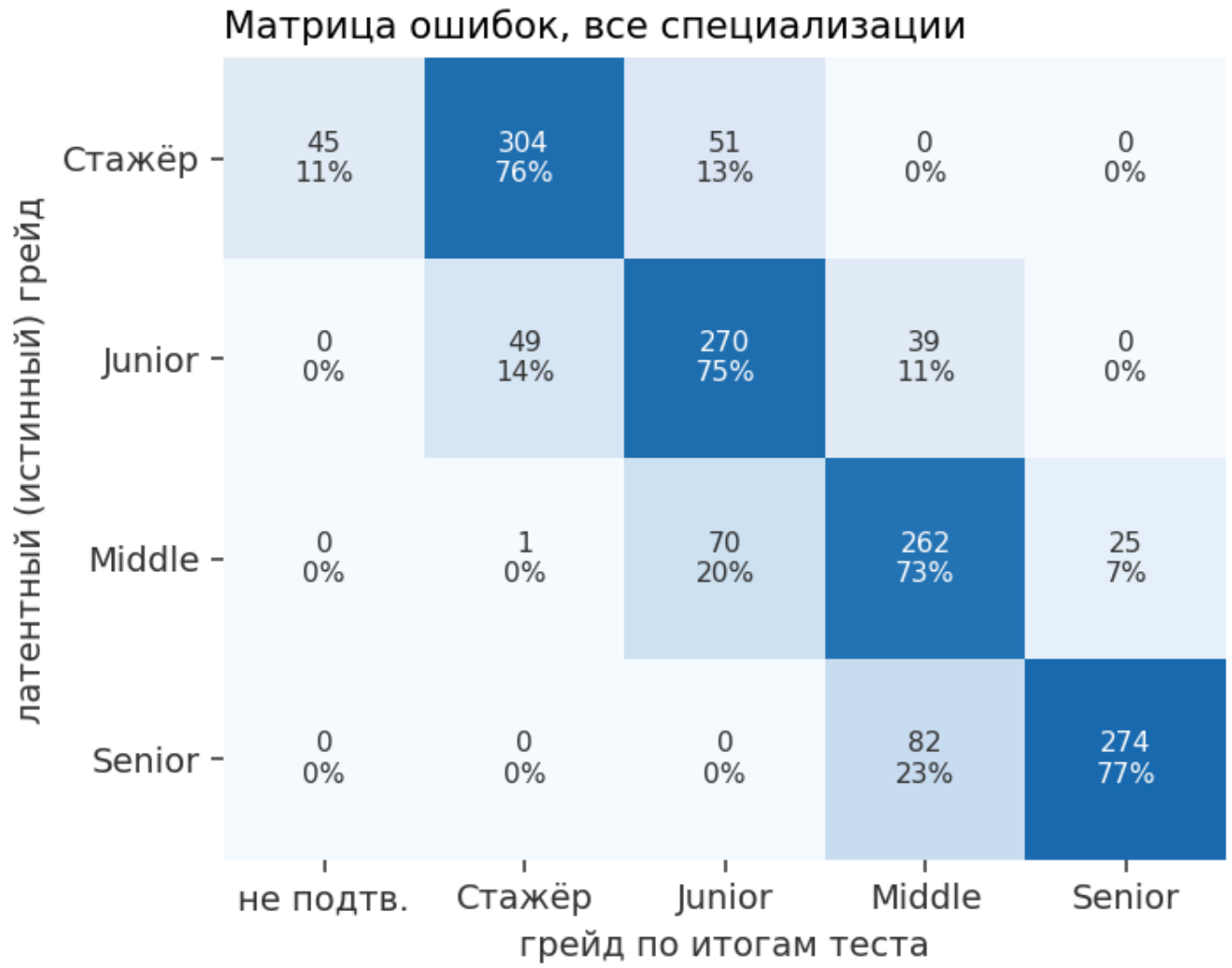
2.10. Кандидаты без истории ФСП

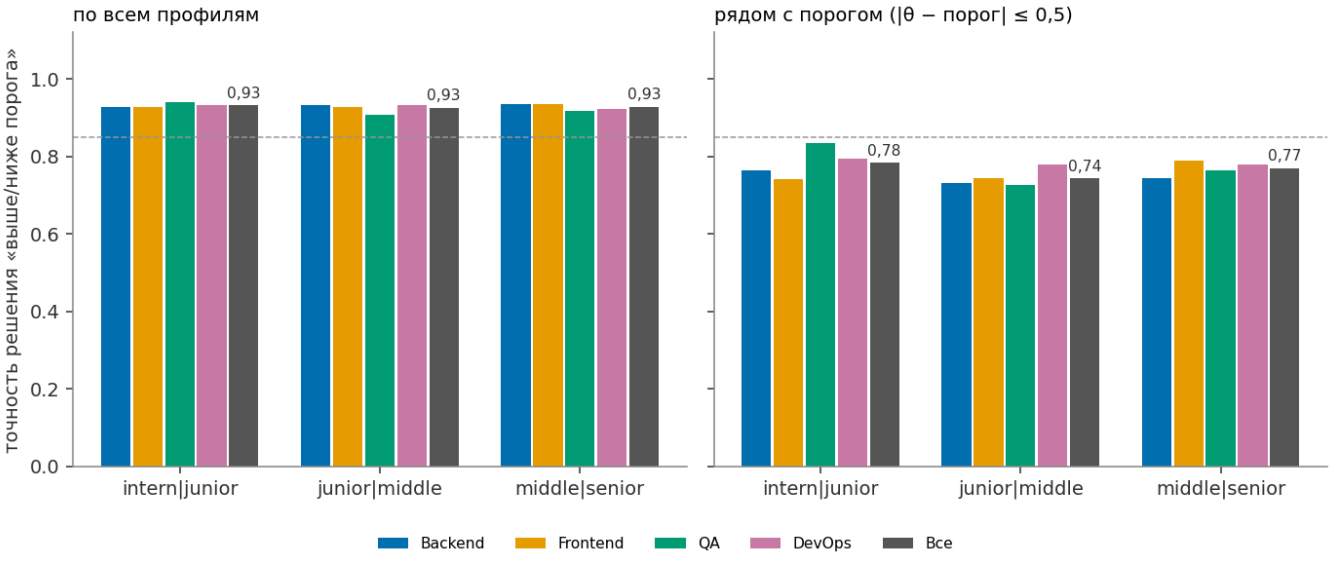
Опрос и тест не принимают на вход ничего из ФСП, поэтому категория присваивается по тем же правилам. Доля подтверждённых тестом: с ФСП — 92,9% (n = 708), без ФСП — 93,3% (n = 1292); разница — только от уровня, который в генераторе коррелирует с ФСП.

2.11. Test-retest и разделяющая способность по специализациям

Отдельный эксперимент на тех же движках (`eval/spec_validation.py`): по 400 латентных профилей на каждую из четырёх специализаций, поровну по грейдам Стажёр / Junior / Middle / Senior, заявка 60% верно / 30% на ступень выше / 10% ниже. Один и тот же профиль проходит три сессии: **A** — основная; **B** — независимая параллельная (другой сид → другие клоны и другая траектория CAT, семейства могут повториться); **C** — пересдача по реальным правилам (spec-testing §7.1: семейства сессии A исключены, то есть другая экспозиция заданий). Истина — латентный уровень профиля; в расчёт точности не входят кандидаты, заявившие грейд ниже истинного: тест на заявленный грейд не может выдать больше, повышение — отдельный тест, это правило продукта, а не ошибка теста.







Test-retest (A↔B — параллельные формы; A↔C — по правилам пересдачи).

Специализация	n	A↔B точно	±1	QWK	r(уровень)	решение сдал/не сдал совпало	к решения	A↔C точно	A↔C QWK	A↔C r(уровень)	общих семейств в C
Backend (Python)	400	68%	99%	0,86	0,87	80%	0,52	69%	0,86	0,85	0,0

Специализация	n	A↔B точно	±1	QWK	r(уровень)	решение сдал/не сдал совпало	к решения	A↔C точно	A↔C QWK	A↔C r(уровень)	общих семейств в C
Frontend (JS/TS)	400	66%	100%	0,86	0,87	79%	0,47	73%	0,88	0,87	0,0
QA	400	68%	100%	0,86	0,86	79%	0,53	63%	0,81	0,79	0,0
DevOps	400	73%	100%	0,89	0,89	80%	0,50	61%	0,80	0,78	0,0
Все вместе	1600	69%	100%	0,87	0,87	80%	0,51	66%	0,84	0,82	0,0

Доля профилей, у которых решение «перевернулось» через порог между сессиями A и B (весь профиль; в скобках — только рядом с порогом, |уровень − порог| ≤ 0,5, где ошибки ожидаемы):

Специализация	Стажёр/Junior	Junior/Middle	Middle/Senior	любой порог	рядом с порогом: грейд разошёлся	SEM по паре сессий
Backend (Python)	9%	11%	10%	32%	37% (n = 299)	0,42
Frontend (JS/TS)	9%	10%	10%	34%	33% (n = 287)	0,42
QA	8%	8%	11%	32%	28% (n = 285)	0,42
DevOps	8%	8%	8%	27%	30% (n = 298)	0,39
Все вместе	9%	9%	10%	31%	32% (n = 1169)	0,41

Разделяющая способность: итог теста против истинного грейда.

Специализация	n	Точный грейд	±1	QWK	Надёжность	SEM (апост.)	SEM у порогов	Заданий	p95
Backend (Python)	371	76%	100%	0,91	0,84	0,45	0,43	16,0	20
Frontend (JS/TS)	369	75%	100%	0,91	0,83	0,47	0,45	16,5	20
QA	360	73%	100%	0,89	0,82	0,47	0,46	17,0	20
DevOps	372	77%	100%	0,91	0,82	0,50	0,48	14,2	15
Все вместе	1472	75%	100%	0,90	0,83	0,47	0,46	15,9	20

Матрица ошибок, все специализации (строки — истинный грейд, столбцы — итог теста):

Истина \ итог	не подтв.	Стажёр	Junior	Middle	Senior
Стажёр	45	304	51	0	0
Junior	0	49	270	39	0
Middle	0	1	70	262	25
Senior	0	0	0	82	274

Точность решения «выше порога / ниже» у каждого порога (всего / рядом с порогом) и AUC оценки уровня между соседними грейдами (ровно «ниже/выше» порога):

Специализация	Стажёр/Junior	Junior/Middle	Middle/Senior
Backend (Python)	93% / 76% / 0,96	93% / 73% / 0,96	94% / 74% / 0,96
Frontend (JS/TS)	93% / 74% / 0,95	93% / 74% / 0,96	94% / 79% / 0,96
QA	94% / 84% / 0,96	91% / 73% / 0,95	92% / 76% / 0,96
DevOps	93% / 79% / 0,97	93% / 78% / 0,98	92% / 78% / 0,96
Все вместе	93% / 78% / 0,96	93% / 74% / 0,96	93% / 77% / 0,96

Информация Фишера, набранная заданиями сессии у порога, и SEM = 1/√I (среднее по профилям):

Специализация	Стажёр/Junior: I / SEM	Junior/Middle: I / SEM	Middle/Senior: I / SEM
Backend (Python)	4,7 / 0,46	4,9 / 0,45	3,5 / 0,53
Frontend (JS/TS)	4,7 / 0,46	4,9 / 0,45	3,3 / 0,55
QA	5,1 / 0,44	4,7 / 0,46	3,2 / 0,56
DevOps	3,9 / 0,51	4,3 / 0,48	3,5 / 0,53
Все вместе	4,6 / 0,47	4,7 / 0,46	3,4 / 0,54

Дискриминация заданий (зачётные задания сессий A; r_pb — ответ против оценки уровня без этого ответа; A_eff — крутизна IRT-кривой из банка; ΔP — прирост доли верных между соседними грейдами):

Специализация	Семейств (n ≥ 25 ответов)	Медиана r_pb	10-й перцентиль	Доля r_pb ≥ 0,20	Медиана A_eff	Медиана ΔP	Флаг r_pb < 0,10
Backend (Python)	43	0,41	0,24	95%	1,35	0,21	нет
Frontend (JS/TS)	39	0,45	0,28	97%	1,35	0,21	нет
QA	25	0,47	0,35	100%	1,35	0,19	нет
DevOps	19	0,52	0,38	100%	1,35	0,21	нет

Эквивалентность клонов. Живых клонов у нас нет, поэтому разброс сложности клона σ_inst (в конфиге 0,15) — допущение. Проверяем чувствительность: как меняются точность и ретест, если клоны на самом деле разнятся меньше или больше (по каждой специализации, доля точного грейда против истины / доля совпадения при ретесте):

Специализация	σ_inst = 0,00	σ_inst = 0,15	σ_inst = 0,30
Backend (Python)	81% / 70%	81% / 68%	79% / 69%
Frontend (JS/TS)	81% / 72%	79% / 72%	82% / 72%
QA	69% / 64%	82% / 78%	75% / 69%
DevOps	75% / 68%	75% / 71%	73% / 63%

Устойчивость к утечке ответов. Коалиция честных участников выкладывает всё, что видела; слабый кандидат (Стажёр/Junior, заявка на ступень выше) вводит слитый ключ чужого клона. Сравнение парное: та же сессия честно и со слитыми ключами. «Осторожный» списывающий, видя другой код, отвечает честно — его прирост равен нулю по построению, поэтому в таблице наивный.

Специализация	n	Коалиция	Банк слит	Прирост оценки уровня [95% ДИ]	Ложный «сдал» честно	со слитыми ключами	Пойман «чужим ключом»	Детектор активен для семейств
Backend (Python)	100	20	80%	-1,76 [-1,91; -1,65]	15%	0%	100%	39 из 55
Frontend (JS/TS)	100	20	98%	-1,71 [-1,83; -1,58]	13%	0%	100%	25 из 41
QA	100	20	100%	-1,83 [-1,95; -1,71]	21%	0%	100%	13 из 27
DevOps	100	20	100%	-1,70 [-1,83; -1,58]	26%	0%	100%	15 из 19

«Двойник в буфере»: покрытие банка. Доля экземпляров, для которых строится двойник (подменённый литерал, ключ двойника ≠ ключа), — детерминированно, без людей и без LLM:

Специализация	Семейств	С двойником	Экземпляров с двойником	Ключ двойника ≠ ключа	Почему нет двойника (экземпляров)
Backend (Python)	55	33	60%	100%	format_unsupported: 48, answer_mode_not_text: 48, language_not_executable: 30, no_candidates: 6, all_candidates_rejected: 1
Frontend (JS/TS)	41	20	48%	100%	format_unsupported: 48, answer_mode_not_text: 48, language_not_executable: 30, executor_failed: 1
QA	27	13	48%	100%	format_unsupported: 30, answer_mode_not_text: 30, language_not_executable: 24
DevOps	19	12	63%	100%	language_not_executable: 24, answer_mode_not_text: 12, format_unsupported: 6

Пилот анти-ассиста (сводка без персональных данных). Пилот 1: 3 модели одного вендора × 2 условия × 2 обычных задания = 12 ответов, верных 12 — правильный ответ ИИ по одному ответу не отличить от сильного человека. Пилот 2: построено 14 заданий с двойником (predict_output — 10, sql_result — 4); ключ двойника ≠ ключа и все гейты пройдены у 14 из 14. **Прогон моделей на этих ловушках не выполнен, поэтому recall двойника не измерен.** Измерен только факт построения двойников (14/14, ключ двойника ≠ ключу) и покрытие банка (ниже, twin). Доля пойманных копипастеров среди живых людей и моделей не измерена; обход скриншотом двойник не ловит.

Выводы по 2.11 (считаются по числам выше).

- Разделяющая способность: точный грейд против истины — 75%, ±1 — 100%, QWK 0,90. Лучше всего DevOps (77%), слабее всего QA (73%). Грубые ошибки (через ступень) редки; ошибаются соседние грейды — у порога.
- Test-retest: точное совпадение грейда между независимыми сессиями 69%, между сессией и пересдачей по правилам 66%; слабее всего Frontend (JS/TS) (66%). Решение «выше/ниже порога» повторяется в 80% случаев. При пересдаче по правилам слабее всего DevOps (61%): в малых банках (QA, DevOps) после исключения семейств первой сессии остаётся меньше подходящих по сложности заданий. к решения около 0,5 — умеренная. Точное совпадение при 4–5 уровнях и ~16 заданиях не может быть близким к 100%; расхождения сосредоточены у порогов.
- Утечка: у наивного списывающего слитые ключи не дают прироста (верхняя граница 95% ДИ прироста оценки уровня по специализациям: -1,58 ≤ 0). Это модельный результат при допущении σ_inst и уникальных клонов; утечку **идеи** семейства он не покрывает (см. 2.6, CUSUM).
- Двойник строится не везде: доля экземпляров с двойником ниже 50% у Frontend (JS/TS), QA — остальные семейства закрыты только клонами и временем ответа.
- Семейств с r_pb < 0,10 нет.
- Всё это симуляция: модель ответа — Раш с экспертными сложностями, а не живые люди; точность против истины отражает процедуру, а не реальную валидность заданий. Живой ретест на людях не проводился.

3. Категоризация по специализации (V-CAT)

300 синтетических профилей опроса (10 специализаций × 30) и 30 трудных случаев. Ответы порождает своя матрица в `eval/vcat.py` (шум 10%, «амбициозные» отметки соседней специализации у 15% профилей, выбор кандидата совпадает с истиной в 80%); классифицирует прод-функция `app.testing.survey.classify`.

Метрика	Подсказка классификатора	Только выбор кандидата
macro-F1	0,93	0,80
Точность	0,93	0,79

Подсказка показывается 16% профилей, верна в 92% случаев; из тех, кто выбрал не свою специализацию, верную подсказку получают 70%.

Специализация	Точность	Полнота	F1
Backend (Go)	1,00	0,87	0,93
Backend (Java/Kotlin)	1,00	1,00	1,00
Backend (Python)	0,88	1,00	0,94
Data Science / ML	1,00	1,00	1,00
DevOps / SRE	0,86	1,00	0,92
Frontend (JS/TS)	0,73	1,00	0,85
Fullstack	1,00	0,67	0,80
Информационная безопасность	1,00	0,90	0,95
Mobile	1,00	0,90	0,95
QA / тестирование	1,00	1,00	1,00

Трудные случаи:

Тип	n	Верная подсказка (top-1)	Подсказка показана
Fullstack	10	0,60	0,00
смена профессии	10	1,00	0,00
противоречивый опрос	10	0,80	0,90

Веса подсказки подбирались не на основном сиде. Прежние веса «технологии / бонус за выбор» (0,3 / 0,5) заменены на (1,2 / 0,8) по сетке на четырёх других популяциях профилей (сиды 11, 12, 13, 14); проверка — на четырёх отложенных (сиды 21, 22, 23, 24), которых в подборе не было, и на основном сиде (таблица выше — уже с новыми весами).

Выборка, веса	Профилей	macro-F1	Подсказка показана	Подсказка верна	Верная подсказка при ошибочном выборе	Трудные случаи: top-1
подбор, прежние веса	1200	0,87	11%	0,79	0,46	0,75
подбор, текущие веса	1200	0,93	14%	0,90	0,68	0,81
отложенные, прежние веса	1200	0,87	13%	0,84	0,47	0,76
отложенные, текущие веса	1200	0,92	17%	0,94	0,71	0,82

Оговорка: генератор профилей писала та же команда, поэтому проверка на отложенных сидах показывает отсутствие подгонки под 300 профилей, а не перенос на живых людей — это нужно перепроверить на реальных опросах. Подсказка ничего не решает: специализацию выбирает кандидат, категорию подтверждает тест.

Сетка «вес технологий × бонус за выбор» на чётных профилях основного сида (проверка на нечётных): текущие (1,2 / 0,8) — macro-F1 на проверочной половине 0,94; лучшие на чётных 1,2 / 0,8 — 0,94.

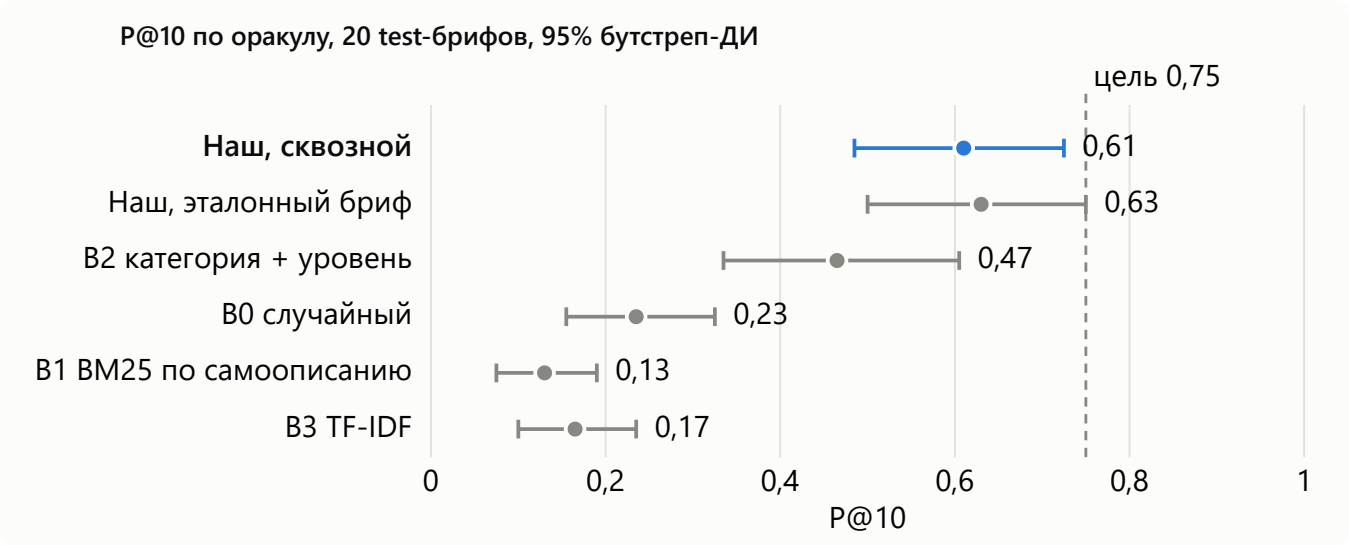
4. Подбор

Оракул релевантности по скрытой истине (VALIDATION_PLAN §1.5): rel = 0 при несмежной специализации; иначе 3, −1 за соседний грейд (≥ 2 ступени → 0), −1 если покрыты не все must, −2 если покрыто меньше половины, −1 если минимум кандидата выше 1,1 · верх вилки. Релевантен ⇔ rel ≥ 2. Навык «покрыт», если он есть в истинном

стеке и (не тестируется или истинный уровень его модуля не ниже порога грейда брифа − 0,5). Формат работы оракул не учитывает.

Популяция — 2 000 кандидатов, из них с подтверждённой категорией 1 864. Сквозной режим: разбор брифа → один уточняющий вопрос, если парсер его задал (отвечает работодатель — вариант из эталона) → обязательная вилка (если парсер её не нашёл, работодатель вводит её в форме) → рекомендованные категории → версия подборки с пресетом «Баланс».

4.1. Главная таблица (20 test-брифов, среднее и 95% бутстреп-ДИ по брифам)



Система	P@10	P@5	nDCG@10	Recall@50	MRR
Наш (сквозной: разбор брифа + 1 уточнение)	0,61 [0,48; 0,72]	0,57 [0,44; 0,69]	0,50 [0,42; 0,57]	0,23 [0,17; 0,29]	0,64 [0,49; 0,79]
Наш (эталонный бриф)	0,63 [0,50; 0,75]	0,58 [0,45; 0,71]	0,51 [0,43; 0,59]	0,24 [0,18; 0,30]	0,67 [0,51; 0,82]
Наш (только парсер, без уточнения)	0,61 [0,48; 0,72]	0,57 [0,44; 0,70]	0,50 [0,43; 0,57]	0,23 [0,17; 0,29]	0,64 [0,49; 0,79]
B2 категория + точечная оценка уровня	0,47 [0,33; 0,60]	0,48 [0,33; 0,64]	0,38 [0,30; 0,47]	0,14 [0,11; 0,16]	0,73 [0,57; 0,89]
B0 случайный порядок в тех же категориях	0,23 [0,15; 0,33]	0,25 [0,16; 0,35]	0,23 [0,17; 0,30]	0,10 [0,07; 0,12]	0,41 [0,27; 0,55]
B1 BM25 по самоописанию (как поиск по резюме)	0,13 [0,07; 0,19]	0,12 [0,06; 0,19]	0,14 [0,10; 0,19]	0,07 [0,05; 0,09]	0,24 [0,14; 0,37]
B3 TF-IDF «бриф ↔ самоописание» (заменитель эмбеддингов)	0,17 [0,10; 0,23]	0,15 [0,09; 0,21]	0,16 [0,12; 0,20]	0,06 [0,05; 0,08]	0,32 [0,21; 0,45]

Потолок P@10 (если поставить всех релевантных наверх): 1,00. Recall@50 ограничен сверху величиной 50 / (число релевантных в популяции), а релевантных у брифа — сотни, поэтому он мал у всех систем.

Разница P@10 с B2 «категория + точечная оценка уровня» (парный бутстреп по брифам):

Система	Разница	95% ДИ	Значимо
Наш (сквозной: разбор брифа + 1 уточнение)	0,14	[-0,01; 0,30]	нет
Наш (эталонный бриф)	0,17	[0,01; 0,33]	да
Наш (только парсер, без уточнения)	0,14	[-0,01; 0,30]	нет
B0 случайный порядок в тех же категориях	-0,23	[-0,34; -0,12]	да
B1 BM25 по самоописанию (как поиск по резюме)	-0,34	[-0,49; -0,20]	да

Система	Разница	95% ДИ	Значимо
B3 TF-IDF «бриф ↔ самоописание» (заменитель эмбеддингов)	-0,30	[-0,47; -0,15]	да

Все 40 брифов (test + dev; ни те ни другие не использовались при настройке формулы). P@10 нашего сквозного режима 0,66 [0,57; 0,74], B2 0,52 [0,42; 0,62]; разница с B2 0,14 [0,04; 0,23] — значима. Против случайного порядка в тех же категориях: 0,36 [0,28; 0,45]. Добавленный pooled-срез отмечен как постфактум: заявленная метрика остаётся на 20 test-брифах.

По оракулу наш подбор не обгоняет B2 — так и пишем. Оракул видит только грейд, стек и деньги; B2 сортирует основную категорию ровно по тому, что оракул вознаграждает сильнее всего (уровень), а наша формула добавляет то, чего оракул не видит: формат работы, свежесть задач и ФСП. Где формула выигрывает — ниже: устойчивость к накрутке навыков и к пересдаче (4.4), проверяемые объяснения (4.6); где проигрывает — разобрано в диагностике выше и в 4.5.

Откуда нерелевантные места в топ-10 (описательная диагностика после прогона, порогов у неё нет):

Причина по оракулу	Наш подбор	B2
грейд совпал, покрыто меньше половины must	41	32
грейд на ступень выше брифа, покрыты не все must или минимум выше вилки	27	38
грейд на ступень ниже брифа, покрыты не все must или минимум выше вилки	8	4
грейд совпал, покрыты не все must и минимум выше вилки	1	27
другая специализация	1	0
грейд и must подходят, но формат/город или зарплата не подходят	0	4
грейд дальше чем на ступень	0	2
пустые места (в категориях меньше 10 кандидатов)	0	0
всего нерелевантных мест из 200	78	107

Два главных источника: (1) must-навыки, которые тест не проверяет (Docker, Kafka, Redis и т. п.): оракул знает истинный стек, а формула по построению видит только доказательства — тест, задачи, ФСП — и не верит самоописанию; (2) кандидаты на ступень выше брифа: в категорию попадают сильные люди, заявившие грейд ниже своего и не повысившие его, а внутри категории Score растёт вместе с результатом теста. Оракул считает «переросших» релевантными, только если покрыты все must. Это расхождение формулы и оракула, а не ошибка движка; вопрос передан владельцу подбора (штрафовать ли «переросших» внутри категории, как показывать непроверяемые must).

4.2. Абляции

Снятый компонент отдаёт вес тесту (ограничения на веса соблюдены). «Без нижней границы» — ранжирование по точечной оценке уровня (SE = 0, без старения).

Вариант	P@10	nDCG@10	Разница P@10 с полной формулой	95% ДИ
без нижней границы (точечная оценка, SE = 0)	0,60 [0,48; 0,72]	0,50 [0,42; 0,59]	-0,01	[-0,04; 0,03]
без K (стек)	0,60 [0,48; 0,71]	0,48 [0,41; 0,56]	-0,01	[-0,03; 0,02]
без F (ФСП)	0,65 [0,53; 0,76]	0,52 [0,44; 0,59]	0,04	[0; 0,09]
без A (свежесть)	0,60 [0,47; 0,72]	0,48 [0,40; 0,56]	-0,01	[-0,04; 0,02]

Вариант	P@10	nDCG@10	Разница P@10 с полной формулой	95% ДИ
без U (условия)	0,60 [0,48; 0,71]	0,49 [0,41; 0,56]	-0,01	[-0,03; 0,01]

4.3. Разбор брифа

Метрика	test	dev (справочно)	Цель
Специализация top-1	0,90	1,00	≥ 0,90
Грейд точно / ±1	0,95 / 1,00	1,00 / 1,00	≥ 0,75 / ≥ 0,95
Навыки: точность / полнота / F1	0,99 / 0,98 / 0,98	0,99 / 0,99 / 0,99	F1 ≥ 0,85
must: F1	0,83	0,86	—
must/nice определены верно (среди найденных)	0,81	0,84	≥ 0,80
Верная категория в топ-3	0,95	1,00	≥ 0,95
Вилка: точно от и до / на руки или до НДФЛ	1,00 / 1,00	1,00 / 1,00	—
Формат точно / город	0,95 / 1,00	1,00 / 1,00	—
Задан уточняющий вопрос	0,00	0,00	—

Ошибки парсера на test-брифах:

Бриф	Стиль	Специализация	Грейд (парсер / эталон)	Пропущены навыки	Лишние	Ошибка must/nice
b05	chat	верно	junior / junior	—	layout	css, responsive
b07	formal	ошибка (backend_python)	middle / middle	—	—	drf
b15	long	верно	senior / senior	highload	—	asyncio, caching, docker, fastapi, kafka, postgresql, redis
b17	chat	верно	junior / junior	—	—	mongodb, postgresql
b21	chat	верно	senior / senior	python	—	—
b23	formal	верно	senior / senior	—	—	webpack
b25	long	ошибка (backend_python)	senior / senior	db_design	—	celery, react, typescript
b29	chat	верно	junior / junior	—	—	jquery, layout
b31	mixed_en	верно	middle / middle	—	—	typescript
b33	formal	верно	junior / intern	—	—	—
b35	long	верно	junior / junior	—	—	css, figma, flexbox, javascript, layout

4.4. Устойчивость

Накрутка навыков. В каждом test-брифе 50 самым слабым кандидатам выдачи добавили в самоописание 5 навыков брифа (1000 случаев). Наш подбор: средний сдвиг вверх 0,0 позиции (максимум 0), ни один не попал в топ-10: заявленные навыки в Score не входят и служат только последним ключом сортировки при равном Score (DECISIONS п.1). BM25 по самоописанию: сдвиг вверх 522,5 позиции, в топ-10 попали 12,4%.

Стабильность при пересдаче (тест пересимулирован на непересекающихся семействах, overlap@10, 20 брифов): наш подбор 0,28, тот же подбор по точечной оценке 0,27, B2 «категория + точечная оценка» 0,06. Исходная цель ≥ 0,70.

Исходный порог не достигнут и, как показывает разбор ниже, **поставлен некорректно** (постфактум-пересмотр, VALIDATION_PLAN §7; сам порог в таблице «Итоги» оставлен и показан как не выполненный). Причина в измерении, а не в формуле: в категории размах истинного уровня — одна полоса грейда, а SE теста из ~15 заданий сопоставим с ней, поэтому надёжность оценки уровня внутри категории — всего 0,25 (по популяции в целом 0,86, но это почти целиком различия между грейдами). Две сессии одного человека честно ранжируют категорию по-разному, и у любого ранжирования, опирающегося на одну оценку теста, состав десятки из сотен кандидатов меняется (Goldstein & Spiegelhalter, 1996, о рейтингах с ошибкой измерения).

Что измеряем вместо «десятка осталась той же»:

Показатель	Значение
overlap@10 нашего подбора	0,28
потолок overlap@10 для ранжирования по одному измерению уровня при надёжности внутри категории 0,25 (симуляция)	0,09
наш overlap@10 / потолок	3,3
индекс Кунчевой@10 (overlap с поправкой на случайность: 0 — как у случайной десятки, 1 — совпадение)	0,26
RBO (p = 0,9, глубина 50; вес позиции убывает, Webber и др., 2010)	0,23
P@10 по оракулу: первая сессия → после пересдачи	0,61 → 0,68
доля прежней десятки, которая остаётся в топ-50 после пересдачи	62%
доля прежней десятки, ушедшей из категории (грейд при пересдаче изменился)	12%

Вывод: состав десятки при пересдаче меняется сильнее, чем хотелось бы, но в 3,3 раза устойчивее потолка «сортировки по оценке теста» (за счёт стабильных K, A, F и U), а **качество** десятки при пересдаче не падает (P@10 по оракулу после пересдачи не ниже, чем в первой сессии), то есть заменяются одинаково пригодные кандидаты. Попытка стабилизировать порядок «полосами» Score (кандидаты внутри полосы ранжируются по hash, «равные по уровню») заметно не стабилизировала (прирост не больше 0,03 — в пределах шума при 20 брифах): ширина 2 — overlap@10 0,28, P@10 0,61; ширина 5 — overlap@10 0,27, P@10 0,62; ширина 8 — overlap@10 0,31, P@10 0,63 (без полос: 0,28 и 0,61); в продукт не внесено. Что делаем в продукте: версия подборки неизменна и меняется только по действию работодателя, грейд защищён гистерезисом.

Добавлен nice-навык: Жаккар топ-10 до и после — 0,96 в среднем, минимум 0,82.

Веса ±0,05 (компенсация пропорционально остальным; если вес упирается в ограничение wF ≤ 0,15 или wT ≥ 0,30, он остаётся на месте). P@10 при весах «Баланс» — 0,61.

Компонент	Сдвиг	P@10	Падение P@10
T	+0,05	0,61	0,00
T	−0,05	0,60	0,01
K	+0,05	0,62	-0,01
K	−0,05	0,61	0,00
F	+0,05	пропущен	—
F	−0,05	0,62	-0,01
A	+0,05	0,60	0,01
A	−0,05	0,61	-0,01
U	+0,05	0,62	-0,01
U	−0,05	0,61	-0,01

4.5. Экспозиция кандидатов без ФСП (DECISIONS п.11)

Условное отношение: наблюдаемое число релевантных кандидатов без ФСП в топ-k к ожидаемому, если бы они попадали в топ-k так же часто, как все релевантные **того же дециля компонента T** (при равном тесте). 1,0 — паритет; цель 0,8–1,25. Децили считаются по всем кандидатам рекомендованных категорий брифа, а не только по 500 строкам версии. Безусловное отношение (доля без ФСП в топе к их доле среди релевантных) — справочно.

k	Условное	Безусловное	Наблюдаемо / ожидаемо
10	0,92	0,92	73 / 79,2
50	0,94	0,93	332 / 351,4

Доля кандидатов без ФСП среди релевантных — 65%.

4.6. Честность объяснений

Проверено 1000 карточек (топ-50 каждого test-брифа).

Проверка	Доля
Σ вкладов = Score (± 0,1 после округления)	100,0%
Каждый довод «за» и «против» ссылается на существующее доказательство (попытка теста, модуль, задача, запись ФСП, настройки, навык брифа)	100,0%
Отсутствие ФСП никогда не попадает в «против»	100,0%
«За» даны только компонентам выше медианы версии более чем на 1 балл	100,0%
Без доказательства довода №1 Score падает не меньше его превышения над медианой (994 проверок)	100,0%

Контрфакт по компоненту довода №1: A — 7 из 7; F — 17 из 17; K — 91 из 91; T — 879 из 879. Контрфакт: T — теста нет; F — нет ФСП; K и A — убраны именно те модули, задачи и записи ФСП, на которые ссылается довод. Довод «Стек» ссылается на все источники подтверждённых навыков, «Свежесть» — на все задачи окна (v_k и A берут максимум, поэтому одного лучшего источника мало: при исходной версии, где называлось по одному источнику, этот пункт был 0,96 — 34 из 91 по K и 7 из 7 по A не проходили из-за запасных источников). U — настройки, а не доказательство, не проверяется.

4.7. Производительность

Сборка версии подборки (Score + объяснения + ранги) на пуле из 1 864 подтверждённых кандидатов: медиана 67 мс, p95 215 мс, максимум 220 мс (40 брифов). Цель p95 ≤ 300 мс. Замер — на машине разработчика параллельно с другими разделами прогона.

4.8. По брифам (test)

Бриф	Стиль	Категория эталона	Релевантных	Категории подборки	Вопрос	P@10 наш	P@10 B2	P@10 B1
b01	formal	backend_python · middle	160	backend_python · middle	—	0,8	0,2	0,3
b03	short	frontend_js · middle	326	frontend_js · middle	—	0,9	0,9	0,1
b05	chat	frontend_js · junior	78	frontend_js · junior	—	0,8	0,4	0,2
b07	formal	fullstack · middle	30	backend_python · middle	—	0,6	0,1	0,0
b09	chat	backend_python · junior	77	backend_python · junior; backend_python · middle; backend_python · senior	—	0,8	0,3	0,1

Бриф	Стиль	Категория эталона	Релевантных	Категории подборки	Вопрос	P@10 наш	P@10 B2	P@10 B1
b11	mixed_en	frontend_js · middle	35	frontend_js · middle	—	0,4	0,0	0,1
b13	short	backend_python · middle	244	backend_python · middle	—	0,9	0,5	0,0
b15	long	backend_python · senior	162	backend_python · senior; backend_python · middle	—	0,4	0,9	0,1
b17	chat	fullstack · junior	41	fullstack · junior; backend_python · junior	—	0,0	0,0	0,0
b19	mixed_en	backend_python · middle	275	backend_python · middle; backend_python · senior	—	0,6	0,7	0,5
b21	chat	backend_python · senior	142	backend_python · senior	—	0,6	0,9	0,0
b23	formal	frontend_js · senior	82	frontend_js · senior; frontend_js · middle	—	1,0	0,3	0,0
b25	long	fullstack · senior	93	backend_python · senior; fullstack · senior; backend_python · middle	—	0,3	0,5	0,3
b27	short	backend_python · junior	184	backend_python · junior	—	0,4	0,5	0,3
b29	chat	frontend_js · junior	97	frontend_js · junior; frontend_js · intern	—	0,7	0,6	0,0
b31	mixed_en	fullstack · middle	64	fullstack · middle; frontend_js · middle; backend_python · middle	—	0,0	0,1	0,2
b33	formal	backend_python · intern	132	backend_python · junior; backend_python · intern	—	0,9	0,1	0,1
b35	long	frontend_js · junior	249	frontend_js · junior	—	0,7	0,8	0,0
b37	chat	frontend_js · middle	177	frontend_js · middle; frontend_js · senior	—	0,7	0,5	0,2
b39	short	backend_python · senior	163	backend_python · senior	—	0,7	1,0	0,1

4.9. dev-брифы (справочно)

Система	P@10	nDCG@10
Наш (сквозной: разбор брифа + 1 уточнение)	0,70 [0,59; 0,81]	0,56 [0,47; 0,65]
Наш (эталонный бриф)	0,71 [0,60; 0,82]	0,57 [0,47; 0,67]
Наш (только парсер, без уточнения)	0,70 [0,59; 0,81]	0,56 [0,48; 0,66]
B2 категория + точечная оценка уровня	0,57 [0,42; 0,72]	0,47 [0,37; 0,58]
B0 случайный порядок в тех же категориях	0,35 [0,21; 0,48]	0,27 [0,18; 0,37]
B1 BM25 по самоописанию (как поиск по резюме)	0,23 [0,15; 0,33]	0,19 [0,13; 0,26]
B3 TF-IDF «бриф ↔ самоописание» (заменитель эмбеддингов)	0,20 [0,13; 0,28]	0,19 [0,13; 0,25]

5. Набор пар для проверки жюри

`backend/eval/pairs_test.jsonl` — 1 704 строк: для каждого из 20 test-брифов пул из объединения топ-20 систем (наш сквозной, B0, B1, B2, B3) и 10 случайных кандидатов. Одна строка — одна пара «бриф × кандидат» в

JSON. Схема — `backend/eval/pairs_test.schema.json` (JSON Schema 2020-12). Кандидаты вне пула при оценке считаются нерелевантными (обычный пулинг); для P@10 и P@5 глубины 20 хватает.

Поля строки:

- `brief_id`, `brief.text`, `brief.form` (вилка из обязательной формы, если её нет в тексте), `brief.gold` — эталон: специализация, грейд, must, nice, форматы, город, вилка;
- `candidate_id`, `candidate.features` — слепые признаки в форме `app.matching.scoring.CandidateFeatures` (категория, оценка уровня и SE, подбаллы модулей с тегами, задачи, VSS по дисциплинам ФСП, заявленные навыки, минимум зарплаты, форматы, город); `null`, если категория тестом не подтверждена; `candidate.self_description` — самоописание (в Score не входит, нужно только для B1/B3);
- `rel` — релевантность 0–3 по оракулу (релевантен ⇔ rel ≥ 2); `ranks_top20` — место в топ-20 каждой системы пула.

Как пользоваться: ранжируйте кандидатов каждого брифа своей системой (или нашей — порядок в `ranks_top20`) и посчитайте P@k по полю `rel`. Импорт этого файла и своих пар — `POST /api/admin/import`, метрики — `POST /api/admin/eval/pairs`, ранжирование — `POST /api/admin/eval/rank` (раздел 7). Скрытой истины (истинный уровень, истинный стек) в файле нет — только итоговая метка оракула.

Как набор собран. Популяция с латентной истиной (`eval/population.py`) → 40 брифов (`eval/briefs`, эталон разметки написан командой) → для каждого test-брифа пул из топ-20 пяти систем и 10 случайных кандидатов → `rel` 0–3 даёт оракул по скрытой истине (специализация, грейд, доля must/nice в истинном стеке, формат/город — не учитывается); релевантен rel ≥ 2. **Размер** — см. выше; минимального размера организаторы не требуют. **Ограничение:** разметка машинная, не рекрутерская (см. раздел 6).

6. Честные ограничения

- Синтетика проверяет процедуру, а не реальные сложности заданий. Ответы порождает модель Раша с разбросом клонов; нецикличных DINA-персон и живого пилота в этом прогоне нет, раздела «люди» нет.
- Экспертной разметки пар нет: релевантность — оракул по истине генератора. Оракул и генератор писала одна команда, поэтому P@k проверяет согласованность механики с нашей же моделью мира, а не с мнением рекрутера.
- Брифы и эталон к ним написаны командой с ИИ-ассистентом; 10 брифов от посторонних авторов, заявленные в концепции, не собраны. Разбиение dev/test есть, но авторы одни и те же.
- Генератор V-CAT и веса классификатора опроса составлены одной командой — проверка частично круговая.
- B3 «LLM под каждого» — симуляция с допущениями (разброс сложности σ = 0,5, доля битых ключей задана, а не измерена). B3 подбора — TF-IDF, а не эмбеддинги: модель эмбеддингов в прогоне не используется.
- Знание «методов» частично обходит защиту: клоны закрывают слив ответов, но не слив идеи семейства; это измерено в 2.6 и ловится CUSUM лишь при массовом сливе.
- От ИИ-ассистента удалённый тест не защищён, как любой удалённый тест; грейд — решение с уровнем доверия, а не сертификат.
- Точное совпадение грейда при пересдаче не может быть близким к 1 при 4–5 уровнях и ~15 заданиях; стабильность для пользователя обеспечивает гистерезис.
- В банке нет pretest-семейств: r_pb считается по боевым заданиям с оценкой уровня без этого задания.
- Fullstack как производная категория в популяции не порождается (каждый кандидат проходит один тест), поэтому Fullstack-брифы ищут в смежных категориях Backend и Frontend.
- Test-retest и разделяющая способность (2.11) измерены на симулированных профилях: ответы порождает модель Раша с экспертными сложностями и допущенным разбросом клонов, поэтому цифры — верхняя оценка для живых людей, где к шуму добавятся усталость, подсказки и неидеальные ключи. Ретеста на людях не проводилось.
- Покрытие двойниками неполное (2.11): для части форматов (выбор ответа, не исполняемые языки) двойник не строится; recall двойника на моделях и людях не измерен.

- Формат работы и город оракул не учитывает, а наша формула учитывает (компонент U): часть «ошибок» по оракулу — это кандидаты, которых формула честно опустила из-за формата.

Пороговые пункты, которые не выполнены в этом прогоне, перечислены в «Итогах»; пороги плана после прогона не менялись. По итогам предыдущего прогона менялись параметры продукта, а не пороги — на других сидах, чтобы не подгонять под основной: `cusum_h` (6 → 11, раздел 2.7), веса подсказки опроса (раздел 3) и полнота доказательств довода в объяснениях (4.6). Что не выполнено и почему:

- **Точный грейд.** Разница с целью 0,75 в пределах выборочной ошибки (n = 1 810, SE ≈ 0,010): на пяти других сидах (101–105) среднее 0,758 при том же коде, на основном сиде 0,748. Ужесточение правила остановки (P ≥ 0,92 / n_min = 10) даёт +0,0…0,3 п.п. ценой +0,6…1,7 задания основной части (+0,8…1,6 с доборами) — бюджет длины (≤ 17 с доборами) нарушается, поэтому не внесено (`eval/stop_rule_sweep.py` , VALIDATION_PLAN §7).
- **Корреляция оценок уровня при пересдаче.** CAT выбирает задания для решения «выше/ниже порога», а не для оценки уровень у всех: SE у кандидатов, далёких от порога, остаётся большим. На пяти других сидах среднее 0,857; самые длинные варианты правила остановки дают 0,861–0,867 при 17,0–17,7 заданиях. Цель 0,88 несовместима с бюджетом длины ≤ 17.
- **Задержка CUSUM при 50% знающих.** Порог h подобран по ложным карантинам (≤ 5% на отдельных сидах), а задержка ≈ h / (информация на ответ) физически ограничена: при Δb = 0,5 и половине знающих эффект на один ответ мал. Уложиться в 200 ответов можно ценой 12–31% ложных карантинов (h = 5–7). CUSUM остаётся сигналом методисту, а не автоматической блокировкой.
- **P@10 значимо выше B2 на 20 test-брифах.** Разница и её ДИ — в 4.1: при 20 брифах интервал чуть захватывает 0, на всех 40 брифах разница значима. Оракул награждает ровно то, по чему сортирует B2 (уровень), а формат, свежесть и ФСП оракул не видит; подгонять веса под оракул ради значимости не будем.
- **Стабильность топ-10 при пересдаче (overlap@10 ≥ 0,70).** Цель поставлена некорректно: при надёжности оценки уровня внутри категории порядка 0,25–0,3 даже идеальное ранжирование по одному измерению даёт overlap@10 около 0,1 (раздел 4.4). Постфактум-метрики — P@10 после пересдачи и отношение к потолку — выполнены; исходная цель оставлена невыполненной.

7. Набор жюри: как проверить вручную на своих профилях

Организаторы проверяют воспроизводимость и разделяющую способность на своём закрытом наборе синтетических профилей. Есть два пути.

A. Test-retest и разделяющая способность (тест) — `eval/jury_check.py` . Принимает JSONL или CSV с профилями и прогоняет их через боевой CAT-движок (три сессии на профиль: основная, параллельная, пересдача по правилам).

```
cd backend
uv run python -m eval.jury_check eval/jury_profiles_example.jsonl --seed 1 --out eval/out/jury_check.json
```

Формат строки (CSV — те же имена столбцов, заголовок обязателен):

Поле	Значение	Обязательно
`id`	уникальная строка	да
`spec`	`backend_python` · `frontend_js` · `qa` · `devops`	да
`true_grade`	ваш эталон: `intern` · `junior` · `middle` · `senior`	да
`theta`	латентный уровень на шкале движка (пороги: Стажёр/Junior −1, Junior/Middle 0, Middle/Senior +1); нет — берётся из полосы грейда по сиду	нет
`claimed_grade`	грейд, на который профиль проходит тест; нет — равен `true_grade`	нет

Пример: `backend/eval/jury_profiles_example.jsonl` (16 профилей). Вывод — таблица в консоль и JSON (`eval/out/jury_check.json`): согласие грейдов, κ, r(уровень), доли переворотов, матрица ошибок против `true_grade`, точность и AUC у каждого порога, информация и SEM у порогов, дискриминация заданий и грейды по каждому профилю. Фиксированный `--seed` даёт тот же результат. Неверная строка → код 2 и сообщение с номером строки и полем.

B. Подбор: свои пары «бриф × кандидат» — админ-API. Формат строк — `backend/eval/pairs_test.jsonl` (схема `pairs_test.schema.json`, раздел 5): бриф, слепые признаки кандидата (или `features: null` — без теста), `rel` 0–3.

1. `POST /api/admin/import` — загрузить пары жюри (данные в банк платформы не попадают); `GET /api/admin/import` — список загрузок.
2. `POST /api/admin/eval/pairs` — P@5, P@10, nDCG@10 с 95% бутстреп-ДИ по брифам (тот же код, что в `make validate`); без `pairs` берётся последний импорт.
3. `POST /api/admin/eval/rank` — ранжирование пула по брифу нашей формулой Score с объяснениями (CSV — `?format=csv`).

Требуется роль администратора; ключи доступа — в `docs/API.md`.

8. Качество заданий: DIF, дрейф, экспозиция

Монитор справедливости и стабильности заданий (`app/testing/monitoring.py`, прогон — `uv run python -m eval.fairness`, seed 20261005). **Это проверка метода на синтетике:** группы задаём мы, в одну группу кладём заведомо смещённые задания, и отчёт обязан их найти. Выводов о реальных людях здесь нет; пол и возраст платформа не собирает, поэтому группы — ФСП (есть/нет), регион (Москва и Санкт-Петербург / остальные), режим доступности, экспресс-версия.

8.1. DIF: Мантель–Хензель, классы ETS

Сессий в анализе: 4000. Критерий сопоставления — $\hat{\theta}$ сессии в стратах по 0,5 логита; MH D-DIF = −2,35·ln α_MH; A — |Δ| < 1 или различие незначимо, C — |Δ| ≥ 1,5 и значимо больше 1, иначе B. Δ < 0 — задание тяжелее для фокальной группы.

Внедрённые смещённые задания (сдвиг сложности для фокальной группы в логитах):

Группа	Семейство	Сдвиг	Доля верных: опорная / фокальная	MH D-DIF (±SE)	Класс
регионы	`py.format.spec`	0,30	0,65 / 0,54	-1,33 ± 0,35	**B**
регионы	`py.defaults.mutable`	0,60	0,47 / 0,35	-1,59 ± 0,36	**C**
регионы	`http.idempotency.retry_cart`	1,00	0,51 / 0,28	-3,56 ± 0,38	**C**
без истории ФСП	`sql.group.having`	0,60	0,63 / 0,47	-2,34 ± 0,37	**C**

Настоящие семейства банка (без внедрения), все пары «семейство × группа» с достаточным числом ответов (не меньше 30 в каждой группе):

Группа (фокальная)	Проверено пар	A	B	C
без истории ФСП	81	77	4	0
регионы	79	77	2	0
режим доступности	74	74	0	0
экспресс-версия	82	82	0	0
Всего	**316**	**310**	**6**	**0**

Доля класса A среди проверенных: **98%**. При ~316 проверках отдельные B ожидаемы (5% ложных срабатываний на уровне значимости), поэтому в лаборатории B — повод посмотреть, C — повод вывести семейство на ревью.

Различие в уровне между группами при этом не принимается за DIF (impact): средний истинный уровень у кандидатов без ФСП -0,03 против -0,04 с ФСП — МН сравнивает людей одного уровня.

8.2. Монитор дрейфа сложности

По ответам после калибровки оценивается сдвиг сложности Δb (метод моментов при апостериорном уровне отвечавшего без этого ответа, 95% ДИ по информации Фишера). Флаг «дрейф» — Δb ≤ −0,50 (стало легче) и весь ДИ ниже 0 (не меньше 40 ответов). Эксперимент: 12 самых частых семейств «утекли» — 80% кандидатов знают ответы (сложность для них ниже на 1,50); 1912 сессий.

- Найдено утёкших семейств: **12 из 12** (полнота 100%); средний Δb по утёкшим -0,71 при «наивном» ожидании -1,20 (оценка сжата: знающие завышают собственный уровень, и это съедает часть сдвига).
- Ложные флаги среди остальных семейств в прогоне с утечкой: 0 (специфичность 100%).
- Контроль (без утечки): флагов **1** из 44 семейств.

Это дополняет CUSUM (раздел 2.7): CUSUM быстро реагирует на поток, монитор даёт величину сдвига с доверительным интервалом для решения методиста. Ночная задача калибровки пишет число флагов в журнал аудита (`scheduler.calibration`).

8.3. Экспозиция семейств и Симпсон–Хеттер

Специализация backend_python: 44 семейств, целевой максимум r_max = 0,60. Экспозиция — доля сессий, в которых семейство показано (одноразовые клоны делают экспозицию конкретного экземпляра ≈ 0, поэтому контроль нужен на уровне семейства). Контроль Симпсона–Хеттера в движке выключен по умолчанию (`quality.sympson_hetter.enabled`); таблица — симуляция с K_i, подобранными за 8 итераций на одной половине кандидатов и проверенными на другой.

Показатель	Без контроля	Симпсон–Хеттер
Сессий	455	455
Максимум экспозиции	0,68	0,67
Семейств выше r_max	5	2
Доля почти неиспользуемых (<1%)	0,00	0,00
Средняя экспозиция	0,37	0,37
Средняя погрешность (SE) итогового уровня	0,447	0,452
Среднее число заданий	16,27	16,16
Верных решений о пороге	0,818	0,804

Семейств с K_i < 1: 4. Нижняя граница экспозиции — средняя длина теста / размер банка = 0,37: r_max ниже этого числа недостижим, поэтому его выбирают выше. Формула итерации: K_i ← min(1, r_max / P(S_i)), P(S_i) — доля сессий, где задание было выбрано до фильтра K (Sympson & Hetter 1985).

Честно про результат. За несколько итераций на ~450 сессиях контроль сходится не полностью: K_i считаются по P(S_i), а P(S_i) растёт, когда соседние задания «отклоняют» (их место занимают остальные); классическая процедура требует десятков итераций на тысячах сессий. Максимум задают семейства самого большого модуля: квота модуля обязывает показывать их почти каждому. Поэтому контроль в продукте выключен, а `eval.fairness` — способ посчитать K_i под реальный банк перед включением.

9. Симуляция рынка приглашений (квоты и сигнал интереса)

Чтобы выбрать числа для `config/contact.yaml` (квоты, недельный лимит, два сигнала) и честно показать границы механизма, мы сыграли рынок приглашений «компания — кандидат» на синтетических агентах (`eval/market_sim.py`, детерминированно: `numpy.random.default_rng(seed)`). Это имитация с допущениями, а не измерение живого рынка. Опора — Coles, Kushnir, Niederle, «Preference Signaling in Matching Markets» (AEJ: Micro, 2013; NBER w16185): один дефицитный сигнал интереса помогает работникам и числу совпадений, лучше всего на сбалансированных рынках; и Kushnir, «Harmful Signaling in Matching Markets» (FEEM, 2010): при очень похожих предпочтениях сигналы могут навредить.

Модель в двух строках. Полезность кандидата от компании = общая «престижная» часть (вертикальная) + личная (горизонтальная); доля общей части λ. Работодатель видит Score с шумом, кандидат читает не больше 5 приглашений в неделю, отвечает «Да» выше личного порога. Режимы: (а) без лимитов — 30 приглашений в неделю на вакансию; (б) квоты из `config/contact.yaml`; (в) квоты + сигнал: работодатель, открывший карточки, при выборе, кого пригласить, учитывает известную вероятность «Да» у сигналящих (в Score сигнал не входит).

Политика берётся из `config/contact.yaml`: активных приглашений на бриф ≤ 20, новых за 7 дней ≤ 60, пакет ≤ 10, новых приглашений кандидату ≤ 5 в неделю; сигналов 2 за окно ≈ 4 нед., живёт ≈ 9 нед. Прогонов на ячейку: 60 сидов (одна и та же популяция во всех режимах).

Сбалансированный 1:1.

Предпочтения	Режим	Заполнено вакансий	Недель до первого «Да»	Gini приглашений	Доля топ-10% кандидатов	Впустую	Приглашений на вакансию	Выигрыш кандидатов
Разные вкусы (λ=0,2)	(а) без лимитов	96,2%	1,15	0,30	21%	85%	36,9	1,25
	(б) квоты	95,9%	1,44	0,33	27%	69%	7,7	0,82
	(в) квоты + сигнал	96,2%	1,39	0,32	28%	65%	7,4	1,04
Смешанные (λ=0,5)	(а) без лимитов	90,6%	1,70	0,38	25%	89%	44,2	0,97
	(б) квоты	88,6%	2,30	0,45	34%	80%	10,0	0,64
	(в) квоты + сигнал	89,1%	2,26	0,45	34%	78%	9,8	0,79
Похожие вкусы (λ=0,9)	(а) без лимитов	70,2%	3,47	0,39	24%	94%	68,5	0,51
	(б) квоты	62,8%	4,15	0,51	34%	88%	10,9	0,39
	(в) квоты + сигнал	62,4%	4,19	0,51	35%	87%	10,8	0,41

Парная разница «квоты + сигнал» − «квоты» (среднее ± 95% ДИ по сидам):

Предпочтения	Δ заполнено	Δ выигрыш кандидатов	Δ недель до первого «Да»
Разные вкусы (λ=0,2)	0,3 ± 0,2 п.п.	0,224 ± 0,013	-0,06 ± 0,02
Смешанные (λ=0,5)	0,5 ± 0,5 п.п.	0,145 ± 0,009	-0,04 ± 0,03
Похожие вкусы (λ=0,9)	-0,3 ± 0,5 п.п.	0,027 ± 0,005	0,04 ± 0,02

Кандидатов вдвое больше.

Предпочтения	Режим	Заполнено вакансий	Недель до первого «Да»	Gini приглашений	Доля топ-10% кандидатов	Впустую	Приглашений на вакансию	Выигрыш кандидатов
Разные вкусы (λ=0,2)	(а) без лимитов	100,0%	1,03	0,39	26%	78%	32,0	0,63
	(б) квоты	100,0%	1,15	0,34	23%	66%	8,7	0,46
	(в) квоты + сигнал	100,0%	1,12	0,34	23%	61%	8,5	0,59
Смешанные (λ=0,5)	(а) без лимитов	99,3%	1,30	0,47	33%	85%	41,2	0,50
	(б) квоты	96,7%	1,76	0,44	31%	79%	11,7	0,36
	(в) квоты + сигнал	96,7%	1,75	0,44	31%	78%	11,6	0,43
Похожие вкусы (λ=0,9)	(а) без лимитов	80,9%	3,00	0,39	27%	95%	89,6	0,27
	(б) квоты	69,7%	3,80	0,60	43%	90%	15,6	0,21
	(в) квоты + сигнал	69,7%	3,80	0,60	43%	89%	15,6	0,22

Парная разница «квоты + сигнал» − «квоты» (среднее ± 95% ДИ по сидам):

Предпочтения	Δ заполнено	Δ выигрыш кандидатов	Δ недель до первого «Да»
Разные вкусы (λ=0,2)	-0,0 ± 0,0 п.п.	0,132 ± 0,007	-0,02 ± 0,01
Смешанные (λ=0,5)	0,0 ± 0,2 п.п.	0,073 ± 0,006	-0,01 ± 0,01
Похожие вкусы (λ=0,9)	0,0 ± 0,4 п.п.	0,012 ± 0,002	-0,00 ± 0,02

Сколько сигналов (сбалансированный рынок).

Сигналов за окно	Смешанные: заполнено	Смешанные: выигрыш канд.	Похожие: заполнено	Похожие: выигрыш канд.
0	88,6%	0,64	62,8%	0,39
1	88,9%	0,74	62,8%	0,40
2	89,1%	0,79	62,4%	0,41
3	89,2%	0,82	62,7%	0,42
5	89,4%	0,84	62,7%	0,43

Размер пакета приглашений (смешанные вкусы, сбалансированный рынок, без сигналов).

Пакет приглашений в неделю	Заполнено	Впустую	Gini	Недель до первого «Да»
3	83,0%	79%	0,44	3,03
5	86,6%	80%	0,44	2,64
10	88,6%	80%	0,45	2,30

Пакет приглашений в неделю	Заполнено	Впустую	Gini	Недель до первого «Да»
20	89,3%	79%	0,44	2,12
40	89,5%	78%	0,43	2,04
без лимита (30)	90,6%	89%	0,38	1,70

Недельный лимит входящих у кандидата (смешанные вкусы).

Лимит в неделю	Заполнено	Отклонено при отправке	Впустую
2	81,2%	85%	78%
3	85,3%	77%	79%
5	88,6%	63%	80%
10	90,1%	39%	79%
20	90,4%	17%	81%

Когда сигнал не помогает. Наивная стратегия — сигнал на лучшую по полезности компанию без оценки шансов:

Предпочтения	Режим	Заполнено	Выигрыш кандидатов	Недель до первого «Да»
Смешанные (λ=0,5)	квоты	88,6%	0,64	2,30
	квоты + сигнал (по шансам)	89,1%	0,79	2,26
	квоты + сигнал (наивно)	88,8%	0,72	2,30
Похожие вкусы (λ=0,9)	квоты	62,8%	0,39	4,15
	квоты + сигнал (по шансам)	62,4%	0,41	4,19
	квоты + сигнал (наивно)	62,8%	0,40	4,18

Что показала симуляция.

- *Квоты — это обмен заполнения на нагрузку.* На сбалансированном рынке со смешанными вкусами рассылка без лимитов заполняет 90.6% вакансий, квоты — 88.6%, но на вакансию уходит 10 приглашений вместо 44 (нагрузка на кандидатов в 4,4 раза ниже), а доля приглашений впустую 80% против 89%. При очень похожих вкусах потеря заполнения заметнее (70.2% против 62.8%). Концентрацию квоты не лечат: Gini 0.45 против 0.38, доля топ-10% кандидатов 34% против 25% (при малом числе приглашений работодатели чаще выбирают лучших по Score). Выигрыш квот — защита кандидата от потока писем, а не равномерность; выигрыш кандидатов от рассылки без лимитов (0,97 против 0,64) — это цена, которую мы платим осознанно.
- *Сигнал возвращает часть потерянного для кандидата.* Против одних квот на сбалансированном рынке сигнал меняет число заполненных вакансий на: разные вкусы +0.3 ± 0.2 п.п., смешанные +0.5 ± 0.5 п.п., похожие -0.3 ± 0.5 п.п. (парное среднее ± 95% ДИ) — то есть почти не меняет. Зато выигрыш кандидатов растёт: смешанные +0.145 ± 0.009 (до 0,79 против 0,64 у одних квот), похожие +0.027 ± 0.005. Читается так: сигнал улучшает **качество** совпадений для кандидата, а не их число; это согласуется с Coles–Kushnir–Niederle (выигрыш на стороне работников, эффект для фирм неоднозначен).
- *Границы (вредный сценарий).* При очень похожих вкусах (λ=0,9) все хотят одних и тех же компаний, сигнал не помогает ни числу совпадений, ни скорости первого «Да»: выигрыш кандидатов +0.027 ± 0.005 против +0.145 ± 0.009 при смешанных. Сигналы уходят в одни и те же места — это тот случай, о котором пишет Kushnir. Заметного вреда, как в его теоретическом примере, в нашей модели не получилось (у работодателя сигнал лишь меняет порядок внутри квоты), поэтому мы заявляем только «нет пользы при общих вкусах», а не «есть вред». Наивная стратегия (сигнал на самую желанную компанию без учёта шансов) выигрыш кандидата снижает по сравнению со стратегией «по шансам» (0,72 против 0,79 при смешанных вкусах), поэтому продукт подсказывает сигнал только по вакансиям, где Score кандидата высок.

- *Допущения, которые могут не держаться на живых данных:* независимые предпочтения кроме λ, одна вакансия на компанию, фиксированные пороги «Да», нет стратегического поведения работодателей, все кандидаты используют сигналы, рынок закрыт на 8 недель. Размеры квот проверены как чувствительность, а не оптимизированы под единственный сценарий.

Решение по настройкам. значения `config/contact.yaml` оставлены без изменений: пакет 10, активных на бриф 20, недельный лимит кандидата 5, сигналов 2 за окно. Пакет: заполнение растёт с размером пакета и выходит на плато (3 — 83.0%, 10 — 88.6%, 40 — 89.5%), колено на 10: дальше +0.8 п.п. ценой 1.1× приглашений. Недельный лимит: 5 даёт 88.6% заполнения против 90.1% при 10, но отсекает больше рассылки (63% против 39% отклонённых при отправке); 5 — осознанный выбор в пользу кандидата (≈1.5 п.п. заполнения), работодателю это видно в квоте и его можно ослабить настройкой кандидата (3/5/10/без ограничения). Сигналы: 0 → 2 → 5 дают выигрыш кандидатов 0.64 → 0.79 → 0.84; два сигнала берут 73% пятисигнального прироста, а дефицитность (смысл сигнала по Coles и др.) сохраняется, поэтому оставлено 2.

Как воспроизвести

```
cd backend
uv run python -m eval.validate            # полный прогон: этот файл, eval/out/validation.json, графики, набор пар
uv run python -m eval.validate --quick    # дымовой прогон на меньших выборках (eval/out/validation_quick.json)
uv run python -m eval.spec_validation --quick   # только test-retest и разделяющая способность (eval/out/spec_validation.json)
uv run python -m eval.jury_check eval/jury_profiles_example.jsonl   # проверка на своих профилях (раздел 7)
uv run python -m eval.validate --save-db  # дополнительно записать ValidationRun в базу
uv run python -m eval.validate --save-db --from-json eval/out/validation.json  # залить готовый результат без прогона
```

Время по разделам, с: population 15,7, matching 45,4, leak 43,0, hysteresis 4,1, cusum 10,4, calibration 0,2, vcat 0,4, cusum_calib:301 65,6, cusum_calib:302 64,6, cusum_calib:303 70,5, s2:sigma:0.0 6,7, s2:sigma:0.3 5,7, s2:sigma:0.5 5,7, s2:a:1.0 6,0, s2:a:2.0 5,8, baselines:0 11,4, baselines:1 6,5, baselines:2 5,9, baselines:3 6,5, baselines:4 11,1, spec_validation 339,6.